Logging Framework Architecture

Patterns: Chain of Responsibility, Strategy Pattern (Appenders/Formatters), Singleton Pattern | Domain: Infrastructure / Chain of Responsibility

Core Requirements & Features	Core Domain Classes & Interfaces
• **Multi-Level Logging:** DEBUG, INFO, WARN, ERROR log severity hierarchy. • **Pluggable Appenders:** ConsoleAppender, FileAppender, AsyncAppender targets. • **Custom Formatting:** JSONFormatter, PatternFormatter (timestamp, thread, class).	• **Logger:** Main client interface for submitting log messages. • **LogManager:** Singleton configuration registry holding logger instances. • **Appender:** Interface for output destinations (Console, File, Socket). • **Formatter:** Converts LogMessage object to formatted String payload.

Critical Execution Workflow & Method Trace

1. Application code calls logger.info('User logged in', userId).
2. Logger checks if current level (INFO) >= configured Logger Threshold Level.
3. Creates LogMessage object containing timestamp, threadName, level, message, & stackTrace.
4. Logger passes LogMessage to registered LogFilter pipeline (returns ACCEPT / DENY).
5. If ACCEPT, Logger iterates all attached Appenders (e.g., FileAppender, ConsoleAppender).
6. Each Appender passes LogMessage to its configured Formatter (e.g., PatternFormatter).
7. Appender writes formatted string payload to destination target (stdout file stream or async buffer).

Concurrency & Thread Safety Mechanics	Design Trade-offs & Interview FAQs
• **AsyncAppender:** Uses LMAX Disruptor / ArrayBlockingQueue to offload disk I/O from application threads. • **Lock-Free Buffer:** RingBuffer reduces thread contention during heavy logging bursts. • **File Writer Lock:** ReentrantLock on FileAppender write stream prevents corrupted interleaved log lines.	• **Sync vs Async Logging:** Sync logging blocks app threads on slow disk I/O; Async logging risks losing buffered log messages on abrupt JVM crash. • **String Formatting:** Defer string formatting until level check passes to avoid unnecessary memory allocations.